

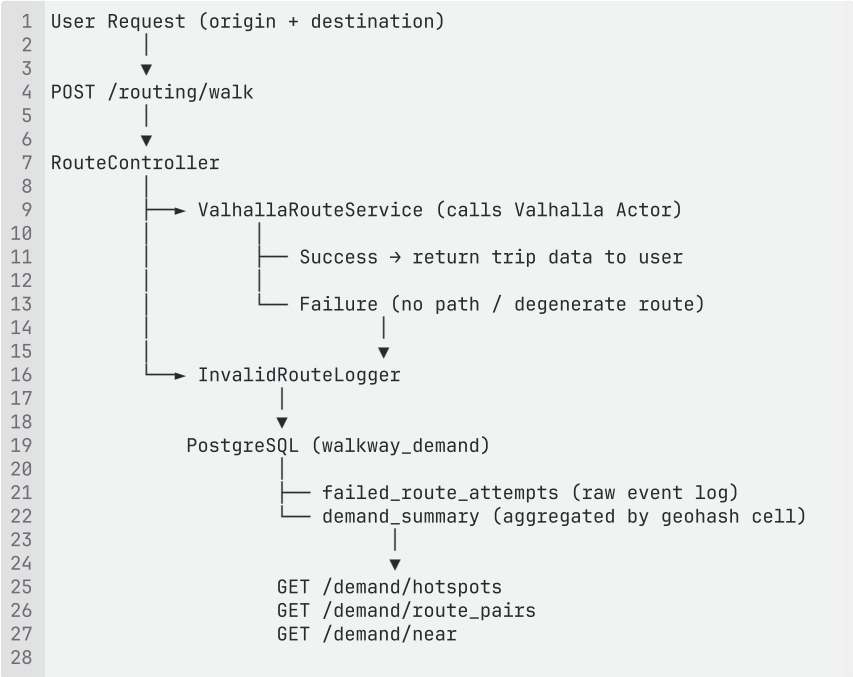
Invalid Route Demand Pipeline — Demo & Setup Guide

Overview

We now have a pipeline that automatically logs failed walking route attempts and spatially clusters them to identify where walkway infrastructure is missing. When a user requests a route and Valhalla can't find a valid pedestrian path, the attempt is logged to a PostgreSQL database with geohash-based spatial clustering so that nearby failed attempts (even 15 feet apart) are grouped together as a single demand hotspot.

This lets us answer the question: **"Where are people trying to walk but can't?"** — and prioritize infrastructure improvements based on real demand data.

Architecture



How Spatial Clustering Works

The pipeline uses **geohashing** to group nearby coordinates into the same grid cell. Each coordinate is encoded into a geohash string, and points within the same cell are automatically clustered.

Current configuration: Precision 9

Precision	Cell Size	Grouping Radius	Use Case
7	~153m × 153m	~500 feet	Neighborhood-level
8	~38m × 19m	~60–120 feet	General walkway gaps
9	~5m × 5m	~15 feet	Current — individual

This means two users tapping spots within ~15 feet of each other will be grouped into the same hotspot, even though their exact coordinates differ.

API Endpoints

POST /routing/walk

Request a pedestrian walking route. Failures are automatically logged.

Request:

```
1 {
2   "origin_lat": 40.4443,
3   "origin_lng": -79.9533,
4   "dest_lat": 40.4460,
5   "dest_lng": -79.9500,
6   "user_id": "user_123"
7 }
8
```

Success Response:

```
1 {
2   "status": "ok",
3   "trip": { ... }
4 }
5
```

Failure Response (logged to database):

```
1 {
2   "status": "no_route",
3   "error_code": 443,
4   "error_message": "Degenerate route: zero length or time",
5   "failure_reason": "degenerate_route",
6   "logged_attempt_id": "10b63779-a372-4b38-aa63-d1b27ce637b1"
7 }
8
```

GET /demand/hotspots

Returns top locations where users keep failing to route, ranked by attempt count.

Parameters:

- `min_attempts` (default: 3) — minimum failed attempts to qualify
- `limit` (default: 50) — max results

Response:

```
1 [
2   {
3     "geohash": "dpqc20n2",
4     "point_type": "destination",
5     "center_lat": 41.00106,
6     "center_lng": -80.49905,
7     "attempt_count": 5,
8     "unique_users": 5,
9     "first_seen": "2026-02-17T14:14:36",
10    "last_seen": "2026-02-17T14:38:14",
11    "failure_reasons": ["degenerate_route"]
12  }
13 ]
14
```

GET /demand/route_pairs

Returns most-requested origin→destination pairs with no valid walkway. Identifies specific missing connections.

Parameters:

- min_attempts (default: 2)
- limit (default: 20)

Response:

```
1  [
2    {
3      "origin_geohash": "dpqc0pvd",
4      "origin_lat": 41.00003,
5      "origin_lng": -80.50008,
6      "dest_geohash": "dpqc20n2",
7      "dest_lat": 41.00106,
8      "dest_lng": -80.49905,
9      "attempt_count": 4,
10     "unique_users": 4
11   }
12 ]
13
```

GET /demand/near

Returns all failed attempts near a specific coordinate (searches cell + 8 neighbors).

Parameters:

- lat (required)
- lng (required)

Failure Reasons

Code	Reason	Meaning
170	no_road_near_destination	No sidewalk/path near the destination point
171	no_road_near_origin	No sidewalk/path near the starting point
442	no_path_found	Sidewalks exist at both ends but no connected route between them
443	degenerate_route	Valhalla returned a zero-length route (no real walkable path)

Project Structure

```
1 app/
2 └─ api/
3   └─ pipeline_api.py           # Existing Valhalla test pipeline
```

```

4 |   | route_api.py           # POST /routing/walk
5 |   | demand_api.py         # GET /demand/hotspots,
  |   route_pairs, near
6 |   | controllers/
7 |   |   | valhalla_controller.py # Existing pipeline controller
8 |   |   | route_controller.py   # Route + failure logging logic
9 |   |   | services/
10 |   |   | route_service.py      # Valhalla Actor integration
11 |   |   | invalid_route_logger.py # Writes to demand pipeline
12 |   |
13 |   | invalid_route_pipeline/    # Core pipeline package
14 |   |   | __init__.py
15 |   |   | schema.py             # DB schema (SQLite + PostgreSQL)
16 |   |   | db.py                 # Backend abstraction layer
17 |   |   | pipeline.py           # Logging, queries, maintenance
18 |   |   | geohash.py            # Spatial clustering utility
19 |

```

Local Setup

Prerequisites

- PostgreSQL 16 (`brew install postgresql@16`)
- psycopg2 (`pip install psycopg2-binary`)
- Valhalla Python bindings (already in project)

Steps

```

1 # 1. Start PostgreSQL
2 brew services start postgresql@16
3
4 # 2. Create the database (first time only)
5 createdb walkway_demand
6
7 # 3. Create the schema (first time only)
8 python -m invalid_route_pipeline.schema --backend postgres --db
  "dbname=walkway_demand"
9
10 # 4. Start the server
11 uvicorn app.main:app --reload
12
13 # 5. Build Valhalla tiles (if not done this session)
14 #   Hit POST /valhalla/test_pipeline in Swagger with:
15 #   {"pbf_file": "Sidewalks.osm.pbf", "lat": 40.4443, "lon":
  -79.9533, "tests": {"route": {}}}
16
17 # 6. Test at http://127.0.0.1:8000/docs
18

```

Useful psql Commands

```

1 # Check logged attempts
2 psql walkway_demand -c "SELECT failure_reason, COUNT(*) FROM
  failed_route_attempts GROUP BY failure_reason;"
3
4 # View demand summary
5 psql walkway_demand -c "SELECT geohash, point_type, attempt_count,
  unique_users FROM demand_summary ORDER BY attempt_count DESC;"
6
7 # Clear test data
8 psql walkway_demand -c "DELETE FROM failed_route_attempts; DELETE FROM
  demand_summary;"
9

```

Shutting Down

```

1 brew services stop postgresql@16
2

```

Cloud Migration

When moving to a cloud database, the only change is the connection string. No code changes required.

```
1 # Local
2 InvalidRoutePipeline(backend="postgres",
3   connection_string="dbname=walkway_demand")
4
5 # Cloud (AWS RDS, Supabase, Neon, etc.)
6 InvalidRoutePipeline(backend="postgres",
7   connection_string="host=db.example.com dbname=walkway_demand user=app
8   password=secret sslmode=require")
```

Schema Management Reference

`create_schema(backend, connection_string, echo=False)`

Creates all pipeline tables and indexes. Safe to call repeatedly — uses `CREATE TABLE IF NOT EXISTS`.

Param	Type	Description
backend	str	"sqlite" or "postgres"
connection_string	str	File path for sqlite (e.g. "walkway_demand.db"), DSN for postgres (e.g. "dbname=walkway_demand")
echo	bool	If True , prints the SQL being executed. Defaults to False .

Demo Test Cases

Successful route (Oakland campus)

```
1 {"origin_lat": 40.4443, "origin_lng": -79.9533, "dest_lat": 40.4460,
2   "dest_lng": -79.9500, "user_id": "test_user_1"}
```

Failed route (outside tile coverage)

```
1 {"origin_lat": 41.5000, "origin_lng": -80.8000, "dest_lat": 41.5010,
2   "dest_lng": -80.7990, "user_id": "test_user_2"}
```

Clustering test (submit all 5, then check /demand/hotspots with min_attempts=3)

```
1 {"origin_lat": 41.0001, "origin_lng": -80.5001, "dest_lat": 41.0011,
2   "dest_lng": -80.4991, "user_id": "test_user_3"}
3 {"origin_lat": 41.0002, "origin_lng": -80.4999, "dest_lat": 41.0009,
4   "dest_lng": -80.4992, "user_id": "test_user_4"}
5 {"origin_lat": 40.9999, "origin_lng": -80.5002, "dest_lat": 41.0012,
6   "dest_lng": -80.4988, "user_id": "test_user_5"}
7 {"origin_lat": 41.0001, "origin_lng": -80.4998, "dest_lat": 41.0010,
8   "dest_lng": -80.4991, "user_id": "test_user_6"}
9 {"origin_lat": 41.0000, "origin_lng": -80.5001, "dest_lat": 41.0011,
10  "dest_lng": -80.4990, "user_id": "test_user_7"}
```